

Untitled 24

Power 모듈 — 2단계 테스트 (12V 스위칭 · 함수 공유)

- 정본(동작 규칙): `revita_link_펌웨어_정리.md` §5.8 — 요청 주체 목록은 본 프로젝트에서 5개로 둔다 (아래 §1.3).
- 연계: 배터리·타이머·LoRa까지 다룬 단계는 `Power_모듈_1단계_테스트_설계.md` 를 본다. 본 문서는 **12V 부스트(Enable) 제어**만 집중해서 검증한다.

1. 12V 스위칭은 어떻게 동작하나

1.1. 하드웨어 쪽

- 보드에 12V 스위칭 전원(또는 부스트)이 있고, **MCU GPIO 한 줄(Enable)** 로 켜고 끈다.
- 초기화 시 Enable은 **꺼진 상태(LOW)** 가 기본이다 (`power_module_init` 에서 핀 설정).

1.2. 논리 쪽 — “누가 하나라도 쓰면 켜고, 아무도 안 쓰면 끈다”

RS485, valve_0, valve_1, button, security 다섯 주체가 동시에 12V가 필요할 수 있다. 그래서 단순 ON/OFF 스위치 하나로는 부족하고, 누가 “나 쓴다”고 했는지를 비트마스크로 모은다.

동작	의미
<code>power_module_request_12v</code> (요청자 비트)	“나(이 주체)는 12V 필요해” → 마스크에 비트를 OR 로 켜다. 마스크가 0이 아니면 Enable 핀 HIGH (12V 공급 쪽이 켜짐).
<code>power_module_release_12v</code> (요청자 비트)	“나는 이제 안 써” → 해당 비트만 끈다. 다른 주체 비트가 남아 있으면 12V는 계속 켜진 채.
마스크가 정확히 0이 되는 순간	Enable LOW — 아무도 안 쓰므로 12V 경로를 끈다.

비유하면 참조 카운팅과 비슷하다. 다만 여기서는 “개수” 대신 **비트(주체 종류)** 로 누가 붙잡고 있는지 표현한다.

1.3. 요청 주체 비트 (`Power12VRequest`) — 5개 고정

12V를 걸 수 있는 호출자는 아래 다섯 가지뿐이다. 각각 서로 다른 비트를 쓴다.

주체	열거형 예시	비트 할당 예 (<code>uint32_t</code>)
RS485	<code>POWER_12V_REQ_RS485</code>	<code>1 << 0</code>
valve_0	<code>POWER_12V_REQ_VALVE_0</code>	<code>1 << 1</code>
valve_1	<code>POWER_12V_REQ_VALVE_1</code>	<code>1 << 2</code>
button	<code>POWER_12V_REQ_BUTTON</code>	<code>1 << 3</code>
security	<code>POWER_12V_REQ_SECURITY</code>	<code>1 << 4</code>

```
typedef enum {  
    POWER_12V_REQ_RS485      = (1u << 0),  
    POWER_12V_REQ_VALVE_0    = (1u << 1),  
    POWER_12V_REQ_VALVE_1    = (1u << 2),  
    POWER_12V_REQ_BUTTON     = (1u << 3),  
}
```

```
POWER_12V_REQ_SECURITY = (1u << 4),
} Power12VRequest;
```

각 모듈은 자기 비트만 request/release 하고, 같은 공개 함수를 쓴다. GPIO는 Power 모듈만 만진다.

펌웨어 정리본 §5.4 예시에는 밸브·시큐리티만 있었으나, 본 제품 계약은 위 5주체로 맞춘다.

1.4. MODULE_STATE_ACTIVE 일 때만

power_run_state 가 INACTIVE이면 request / release 는 마스크와 핀을 바꾸지 않고 return 한다 (§5.11).
본 테스트 문서 범위에서도 ACTIVE 고정으로 두면 검증이 단순하다.

1.5. 조회

- power_module_is_12v_on() — 마스크가 0이 아니면 true.
- (선택) 레지스터 POWER_12V_STATUS_ADDR 등에 기록하는 것은 정본 §5.8과 동일.

2. “함수 공유”가 이 테스트에서 의미하는 것

- 하나의 구현 power_module_request_12v / power_module_release_12v 만 두고, RS485 · valve_0 · valve_1 · button · security 가 같은 API를 서로 다른 비트로 호출한다.
- 테스트에서는 다섯 모듈이 없어도 된다. main 이나 스텝이 비트만 바뀌 가며 request/release 를 호출해, 마스크·핀·전압이 위 규칙대로인지 본다.

즉 “함수 공유” = 12V 핀 제어 코드는 Power 모듈 한 곳, 호출자는 비트만 다르게 공유한다.

3. 무엇을 만들면 되나 (테스트 범위)

- 1단계에서 이미 있는 power_module_init 에 12V Enable GPIO 를 붙인다 (DTS·드라이버는 보드에 맞게).
- power_module_request_12v / power_module_release_12v / power_module_is_12v_on (및 필요 시 is_12v) 구현.
- 내부 정적 변수 v12_request_mask 로 OR/AND~ 동작.
- 테스트용으로 5비트 각각을 한 번씩 건드리는 호출 경로를 두면 좋다 (예: 스텝 5개 또는 main 에서 순서대로 request / release 호출).

배터리 타이머·LoRa는 이 문서의 필수 범위는 아니다 (이미 앞 단계에서 검증했다면 생략 가능).

4. 어떻게 테스트하면 되나

아래는 멀티미터로 12V 레일(또는 Enable 뒤 전압) 을 보며 체크하는 시나리오 예이다.

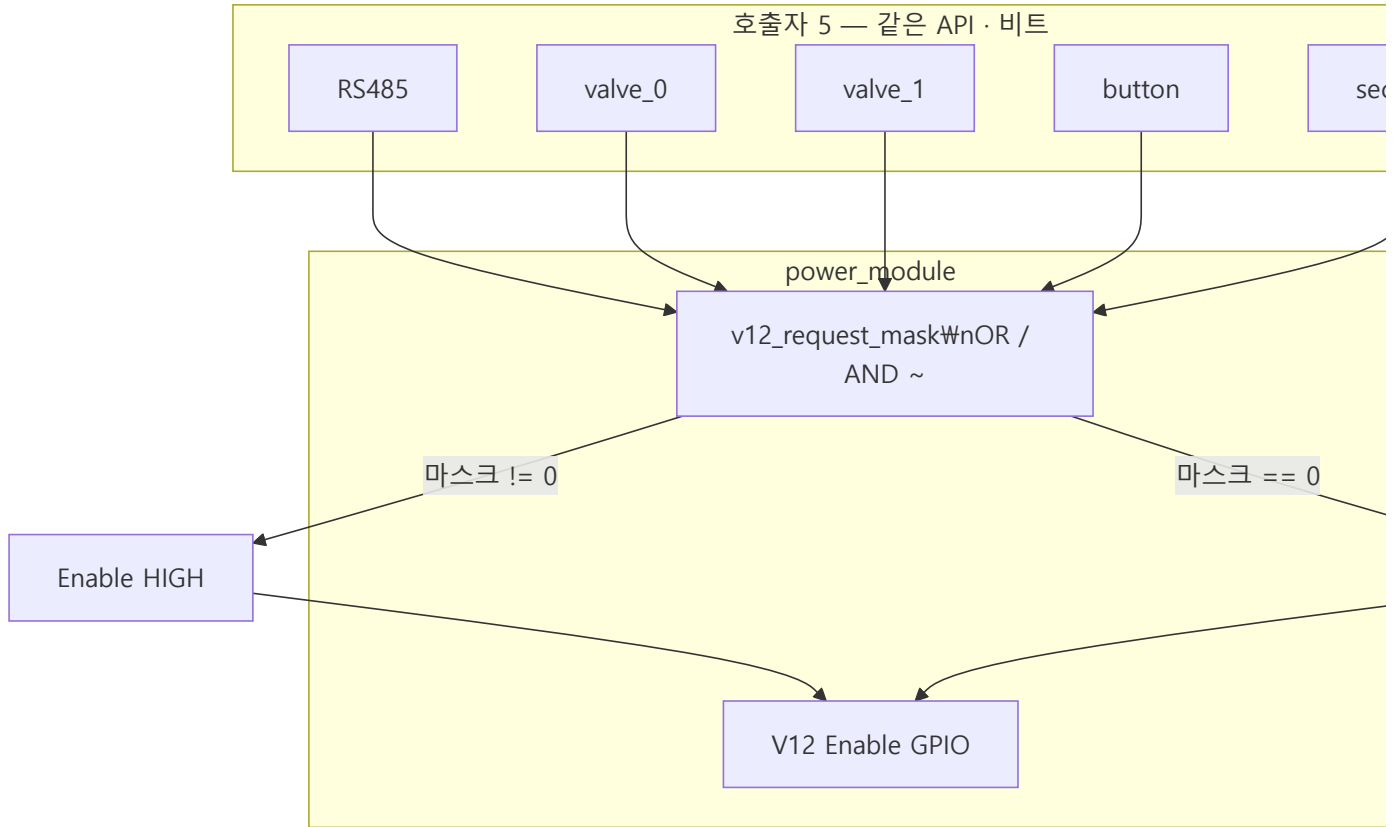
- 부팅 직후 — 12V 꺼짐, is_12v_on() == false .
- request(RS485) — 켜짐.
- request(VALVE_0) 추가 — 여전히 켜짐 (두 비트).
- request(VALVE_1) · request(BUTTON) · request(SEcurity) 순으로 더해 다섯 비트 모두 ON — 레일 유지.
- release 를 한 주체씩 역순(또는 임의 순)으로 해 가며, 마지막 한 비트를 떼기 전까지는 켜짐, 다섯 번째 release 후 마스크 0꺼짐을 본다.

6. 같은 비트로 `release` 두 번 등 — 예외 없이 마스크만 맞으면 됨.

7. (선택) **INACTIVE** 로 두고 `request` 호출 — 핀이 안 바뀌는지.

로그로 `v12_request_mask` 값을 잠깐 찍어 두면 시나리오 디버깅에 좋다.

5. 동작 흐름 (다이어그램)



6. 한 줄 요약

12V 스위칭은 **Enable** 핀 하나를 `request / release` 비트마스크로 `RS485 · valve_0 · valve_1 · button · security` 다섯 주체가 나눠 쓰고, **마스크가 비었을 때만** 끄는 방식이다. 2단계 테스트는 그 공유 함수가 의도대로 동작하는지, 전압·로그로 확인하면 된다.